Start Claude in the right folder

Keep `~/.claude/projects` in order: a session is filed under the folder it starts in

One log line per session, a word to the agent when it starts in a parent folder, and a guard on the terminal command.

Walkthrough session order | Changed 2026-11-06 | For Claude Code on a Mac

You are here: drasticstatic lab › `my-template` › `walkthroughs/` › `CLAUDE-SESSION-ORDER.md` · public · `main` is the working branch

TL;DR

1. **Claude Code files a session under the folder `claude` started in.** Start it in `~` or `~/code` and the session lands in `-Users-<you>` or `-Users-<you>-code`, beside nothing it belongs to.
2. **Nothing can force the folder.** A SessionStart hook cannot stop a session or change where it is filed. It can log the start and tell the agent to ask.
3. **Three pieces, installed by you:** the hook (all apps), a `claude` guard for zsh (the terminal), and the manifest that lists every project folder.
4. **Started in the wrong place anyway?** Type `/cd ~/path/to/the-repo` in the session. Claude Code moves the session there.
5. **Just want the commands?** `startup/CLAUDE-SESSION-START.md` is the short start-up page: what to type, what the guard refuses, how to check and undo.

What Claude Code does, from its own documentation

Read Nov 6, 2026 from code.claude.com/docs/en/hooks, /sessions, /memory and /claude-directory (Claude Code 2.1.319).

Question	Answer
Where does a transcript go?	`~/.claude/projects/<start folder>/<session id>.jsonl`; `<start folder>` is the whole path with every character but a letter or a digit turned into `-`
Where does auto memory go?	`~/.claude/projects/<git repository root>/memory/`, shared by the repository's subfolders and worktrees; outside a repository, the start folder
Can a setting rename project folders?	Not in everyday use. `CLAUDE_CODE_PROJECT_DIR_NAME` names the folder only together with `CLAUDE_CONFIG_DIR` (a separate config directory, made for hosts that embed Claude Code). `autoMemoryDirectory` moves memory only
What does a SessionStart hook receive?	JSON on stdin: `session_id`, `transcript_path`, `cwd`, `hook_event_name`, `source` (`startup`, `resume`, `clear`, `compact`, `fork`), and sometimes `model`
Can it stop the session?	**No.** SessionStart has no blocking: exit code 2 only shows its stderr to you. The general `"continue": false` field is not described for SessionStart, so nothing here relies on it
Can it talk to the agent?	**Yes.** `{"hookSpecificOutput":{"hookEventName":"SessionStart","additionalContext":"..."}}` on stdout, or plain text, goes into the agent's context before the first prompt

Can a session be moved after it starts?	**Yes.** `/cd <folder>` relocates it to that folder's project storage, and `claude --resume` there finds it
How long are sessions kept?	`cleanupPeriodDays`, 30 days by default, by file age. Memory is kept. Set it in `~/.claude/settings.json` if you want transcripts longer

Install the three pieces

The short way: `sh ~/code/my-template/scripts/install-claude-session-order.sh` does steps 1 and 2 below, plus `"cleanupPeriodDays": 3650` so transcripts stop aging out after 30 days. It backs both files up first, is safe to run twice, and `--undo` puts them back. Run it yourself: it changes your own settings.

All three live in `scripts/claude-session-start.sh` 's header too. Prove them first: `sh scripts/claude-projects-manifest.selftest.sh` (a fake home in a temporary folder; your own `~/.claude` is never read).

1. The hook. Add this to `~/.claude/settings.json` (the user file, so it covers every folder). If the file already has a `"hooks"` key, add `"SessionStart"` inside it and leave the other events alone. The file must stay strict JSON.

```
{
  "hooks": {
    "SessionStart": [
      {
        "hooks": [
          { "type": "command", "command": "sh \"$HOME/code/my-template/scripts/claude-session-start.sh\"", "timeout": 10 }
        ]
      }
    ]
  }
}
```

Each start then adds a row to `~/.claude/projects/SESSIONS-LOG.tsv` (date, session id, source, folder). In a parent folder (`~` , `~/code` , `~/dappu` , `~/drasticstaticLabs` , `~/intent/workspaces` , or a folder in `~` that holds repositories without being one), the agent is told to ask which repository you mean and to suggest a restart there or `/cd` . It never blocks and always exits 0. Run `/hooks` in a session to see it registered.

2. The terminal guard. Paste this into `~/.zshrc` , then open a new Terminal tab:

```
claude() {
  local -a pass parents repos kids; local here=0 a d p i is_parent=0
  for a in "$@"; do if [[ $a == --here ]]; then here=1; else pass+=("$a"); fi; done
  case "${pass[1]-}" in
    -p|--print|-v|--version|-h|--help|mcp|config|doctor|update|install|plugin|plugins|purge|auth|agents|attach|stop|rm|setup-token)
      command claude "${pass[@]}"; return ;;
  esac
  d=${PWD:A}
  parents=("$HOME" "$HOME/code" "$HOME/dappu" "$HOME/drasticstaticLabs" "$HOME/intent/workspaces")
  (( ${parents[(Ie)$d]} )) && is_parent=1
  if (( ! is_parent )) && [[ ${d:h} == $HOME && ! -e $d/.git ]]; then
    kids=($d/*/.git(N)); (( ${#kids} )) && is_parent=1
```

```
  fi
  if (( is_parent && ! here )); then
    print -r -- "claude: ${d/#$HOME/~} is a parent folder. A session is filed under the folder it
starts in, so start it in the repository:"
    repos=($d/*/.git(N:h:t)); i=1
    for p in $repos; do print -r -- "  $i) cd ${d/#$HOME/~}/$p && claude"; (( i++ )); done
    (( ${#repos} )) || print -r -- "  (no repositories directly inside; cd into the one you mean)"
    print -r -- "Or start here anyway: claude --here"
    return 1
  fi
  command claude "${pass[@]}"
}
```

Typing `claude` in `~/code` now lists the repositories and stops. `claude --here` starts anyway. Subcommands such as `claude --version`, `claude mcp` and `claude -p` pass straight through. The guard covers the terminal only: VS Code and the desktop app start Claude without a shell, which is why the hook exists.

3. The manifest. After any folder move, and whenever you like:

```
python3 -I ~/code/my-template/scripts/claude-projects-manifest.py          # writes
~/.claude/projects/MANIFEST-LOG.md
python3 -I ~/code/my-template/scripts/claude-projects-manifest.py --check  # exit 1 on an orphan or
an unlisted move
```

It reads only the `cwd` field and the timestamps from the first lines of each session file, never prints what was said, and never edits or moves one. It also lists the sessions about to age out. Folders are moved with `rename-local-repo.py` (SOP-IT-006), which now adds its own line to the manifest's Moves section.

If something goes wrong: the backups

`install-claude-session-order.sh` changes two files, and copies each one before it touches it:

File it changes	Its backup	What the change is
`~/.claude/settings.json`	`~/.claude/backups/settings-before-session-order.json`	adds `"cleanupPeriodDays": 3650` (or keeps a larger number already there) and one SessionStart hook; every other key is kept
`~/.zshrc`	`~/.zshrc.before-session-order`	appends the `claude` guard between its `# >>> claude-parent-guard (zsh)` and `# <<< claude-parent-guard (zsh)` lines

- **The backups are from the first install, and stay that way.** The script copies a file only when its backup does not exist yet, so a second run never overwrites one. `--undo` therefore always returns to the state before the first install.
- **The copies keep the original's dates** (`cp -p`), so the settings backup shows the day `settings.json` last changed before the install, not the day of the install.
- **A bad write cannot land.** The new `settings.json` is written to `settings.json.tmp`, read back as JSON, and only then put in place. If `settings.json` did not exist, the script starts from `{}` and that is what is backed up.
- **The script prints key names, never values** (`settings.json` can hold tokens).
- Claude Code keeps its own backups in the same folder (`.claude.json.backup.<number>`). They belong to Claude Code; leave them alone.

Check that it worked.

```
grep -c claude-parent-guard ~/.zshrc                 # 2: the two marker lines of the guard
python3 -I -c 'import json, os;
print(json.load(open(os.path.expanduser("~/.claude/settings.json"))).get("cleanupPeriodDays"))'   # 3650
```

Then start a new session and type `/hooks`: SessionStart lists `claude-session-start.sh`. In a new Terminal tab, `cd ~/code && claude` lists the repositories and stops.

Undo it. `sh ~/code/my-template/scripts/install-claude-session-order.sh --undo` copies each backup back over its file and says `restored` for each. It puts back the whole file as it was before the first install, so anything added to either file since (a permission you allowed, an alias) goes too: copy the current file aside first if it matters.

Undo it by hand, the same two copies:

```
cp -p ~/.claude/settings.json ~/.claude/backups/settings-$(date +%Y%m%d-%H%M).json   # keep today's, just in case
cp -p ~/.claude/backups/settings-before-session-order.json ~/.claude/settings.json
cp -p ~/.zshrc.before-session-order ~/.zshrc
```

Open a new Terminal tab and start a new Claude session afterwards: both files are read only at start.

A copy of `~/.claude/projects` itself. The backups above cover two settings files, not your sessions. For a nightly, encrypted archive of every transcript and memory file, into iCloud Drive or any folder you choose, see A nightly sealed copy of `~/.claude/projects` (`scripts/claude-projects-archive.sh`).

Workspaces for every nest

Every parent folder now has a VS Code window of its own. What a workspace is, how to open one and how to start Claude in the right repository from it: SOP-IT-006, *Working in VS Code workspaces*.

Nest	Workspace file
`~/code`	`~/code/code.code-workspace`
`~/dappu`	`~/dappu/dappu.code-workspace`
`~/drasticstaticLabs`	`~/drasticstaticLabs/drasticstaticLabs.code-workspace`
`~/littlebirds-git-home`	`~/littlebirds-git-home/littlebirds-git-home.code-workspace`
`~/intent/workspaces` (retired, history only)	`~/intent/intent.code-workspace`, each folder named `retired · <repository> (<branch>) · <Intent workspace>`
each organisation folder	`~/<organisation>/<organisation>.code-workspace`, seven of them

Each file lists the nest's repositories and, where the nest is itself a repository, the parent last as `<name> (parent)`. They describe this machine, so a parent repository gitignores them (`*.code-workspace`). Your own workspace files stay as you made them.

Each nest also has a `MANIFEST.md`: what each child is, where its origin is, its branch, whether it is ahead or behind, and whether the workspace and the `.gitignore` have kept up. `scripts/nest-manifest.py` writes it (`python3 -I ~/code/my-template/scripts/nest-manifest.py ~/dappu`), and `--check` exits 1 when a child is missing from the workspace, a parent

repository does not ignore a child, or the workspace names a folder that is gone. It is read-only on every child. A MANIFEST.md names private repositories, so it never goes in a public one.

Limits, honestly

- The hook speaks to the agent, not to you, and the agent can only ask. The guard is the only piece that stops anything, and only in zsh.
- A parent folder that is itself a git repository (`~/code` , `~/dappu` , `~/drasticstaticLabs` are) shares its memory with every plain folder inside it. Starting in the real repository is the cure.
- Claude Code may change how it files projects. If this page ever seems wrong, re-read the pages named above and re-run the manifest.
- Several repositories in one VS Code window? SOP-IT-006, *Working in VS Code workspaces* says what a workspace is and how to start Claude in the right repository from one.

Change log

Date	Change
2026-11-06	The backups section links the nightly encrypted archive of `~/.claude/projects` (CLAUDE-PROJECTS-ARCHIVE.md).
2026-11-06	TL;DR point 5: the short start-up page in `startup/`.
2026-11-06	*If something goes wrong: the backups* (what the installer copies, `--undo`, a manual restore) and *Workspaces for every nest* (the workspace files, `nest-manifest.py`).
2026-11-06	`scripts/install-claude-session-order.sh`: one line installs the hook, the guard and a ten-year `cleanupPeriodDays` (a helper session is refused this, rightly).
2026-11-06	First version: the hook, the zsh guard and the manifest, written after the labs moved to `~/drasticstaticLabs`. Not installed by default.